

BlasPart: A Deterministic Parallel Partitioner for Balanced Large-Scale Hypergraph Partitioning

Shengbo Tong, Chunyan Pei, and Wenjian Yu

Dept. Computer Science & Tech., BNRist, Tsinghua Univ., Beijing 100084, China

Abstract—Balanced hypergraph partitioning is a fundamental problem in applications like VLSI design, high-performance computing, etc. Nowadays, large-scale hypergraphs become more common due to the increasing complexity of modern systems. Thus, fast and high-quality deterministic partitioning algorithms are largely in demand. Regarding the quality of partitioning, balance is a critical concern when the number of partitions increases. In this paper, we propose BlasPart, a deterministic parallel algorithm for balanced large-scale hypergraph partitioning. BlasPart leverages a recursive multilevel bisection framework to achieve high-quality partitions while ensuring deterministic outcomes. A level-dependent balance constraint is also proposed to further improve the efficiency and effectiveness of the proposed partitioner. Extensive experiments, with comparisons to the state-of-the-art partitioners (hMETIS, BiPart, and Mt-KaHyPar-SDet), demonstrate that BlasPart achieves better balance and scalability while maintaining competitive partitioning quality and efficiency. BlasPart runs $3.33\times$ faster than Mt-KaHyPar-SDet on average for a 4096-way partitioning task on six benchmarks.

Index Terms—balanced partitioning, deterministic parallelism, large-scale hypergraph partitioning, multilevel bisection

I. INTRODUCTION

Balanced hypergraph partitioning is a well-known NP-hard problem [1–3] and has been investigated in many research domains. The goal of balanced hypergraph partitioning is to divide the vertices of a hypergraph into blocks of approximately equal size while minimizing the number of hyperedges that span multiple blocks. Many heuristic algorithms [4–8] have been proposed and achieved relatively good performance.

In the field of Electronic Design Automation (EDA), balanced hypergraph partitioning has been widely used in various scenarios for VLSI circuit design [9, 10]. The growing demand in the digital electronics market and intense competition among enterprises drive the need for highly integrated Systems on Chip (SoC), i.e. VLSI circuits containing billions of elements. The circuit with the connections among elements can be naturally modeled as a hypergraph. Thus, the partition of circuits is equivalent to a hypergraph partitioning problem, which is considered in various stages of VLSI design, e.g. logical design, physical design, and circuit simulation, etc [11, 12]. Specifically, the emerging cloud-based EDA, parallel processing, 3D integration, and FPGA-based emulation request the support of balanced hypergraph partitioning, so as to efficiently balance workloads and minimize interconnections. Hypergraph partitioning also has numerous applications in

many other areas besides EDA, including linear algebra [13], boolean satisfiability [14], as well as storage sharding [15].

As the scale of the problem grows, the need for fast, high-quality partitioning algorithms for large-scale hypergraphs becomes more urgent. Parallelism is a common approach to accelerate partitioning, and several parallel hypergraph partitioners [16–22] have been developed. However, parallelism often introduces non-determinism, which is undesirable in some scenarios like VLSI design because it may result in redoing all the following time-consuming operations. Moreover, larger hypergraphs mean more partitions as well, which makes achieving good balance more challenging.

In response to the aforementioned challenges, we propose a deterministic parallel algorithm for Balanced large-scale hypergraph Partitioning, named **BlasPart**. The main contributions and results of this work are as follows.

- 1) A recursive multilevel bisection framework is proposed for parallel k -way hypergraph partitioning. It combines comprehensive treatments for efficient serial and parallel bisection of hypergraphs, so as to achieve high-quality partitions while ensuring deterministic outcomes.
- 2) A level-dependent balance constraint for bisection tasks is proposed for the multilevel bisection framework. It enables more efficient and effective hypergraph partitioning while achieving well-balanced results or meeting the user-specified imbalance ratio for large-scale hypergraph partitioning (with a large value of k as well).
- 3) The proposed techniques are integrated into the partitioner named BlasPart. Extensive experiments on large hypergraphs show that BlasPart achieves better balance and scalability while maintaining competitive partitioning quality and efficiency, compared to the state-of-the-art parallel deterministic partitioner Mt-KaHyPar-SDet [22].

II. PRELIMINARIES

A. Hypergraph Partitioning

A hypergraph is a generalization of a graph where each edge can connect more than two vertices. In a hypergraph $H = (V, E)$, V is the set of vertices and E is the set of nonempty subsets of V called hyperedges. The goal of balanced hypergraph partitioning is to divide the vertices into k blocks (partitions) such that the number of vertices in each block is approximately equal, while minimizing the number of hyperedges across at least two blocks (*cut-net*). The

This work is supported by Beijing Natural Science Foundation (No. Z230002). The executable program of BlasPart and running instructions are available at <https://numbda.cs.tsinghua.edu.cn/download.html>.

balance of the result of k -way hypergraph partitioning can be measured by the imbalance ratio:

$$\delta = \max_{1 \leq i \leq k} \frac{||V_i| - \frac{|V|}{k}|}{\frac{|V|}{k}}, \quad (1)$$

where $|V_i|$ is the number of vertices in block i , and $|V|$ is the total number of vertices. Some hypergraph partitioners, like hMETIS [4] and TritonPart [11] take the following constraint

$$(1 - \epsilon) \frac{|V|}{k} \leq |V_i| \leq (1 + \epsilon) \frac{|V|}{k}, \quad \forall 1 \leq i \leq k, \quad (2)$$

into the problem formulation. Thus, they can produce well-balanced partitioning results, guaranteeing that imbalance ratio $\delta \leq \epsilon$. Some other partitioners like PaToH [5] and KaHyPar [8] only take the upper bound in (2) as a constraint, which makes their result's imbalance ratio possibly very large.

The objective function of the hypergraph partitioning is generally called *cutsize*, which may be different from *cut-net* in some scenarios. For example, for the hypergraph partitioning aimed at the applications of MFS-based emulation or parallel computing, slightly different metrics like *connectivity* [23] and *soed* [24] are used as the cutsize for minimization. In this work, we use the traditional *cut-net* definition [25], where each cut hyperedge only contributes 1 to the cutsize. Moreover, we pursue the balanced partitioning result with a small imbalance ratio, as it suits more applications, like workload balance in large-scale parallel computing.

B. Related Work

1) *Serial Hypergraph Partitioning*: There are a lot of serial hypergraph partitioning algorithms mostly based on heuristic methods. One type of partitioner employs the multilevel paradigm, which consists of coarsening, initial partitioning, and refinement, and was first proposed by hMETIS [4] and followed by PaToH [5] and UMPa [6]. KaHyPar [8] further improves the partitioning results by introducing preprocessing and flow-based techniques. TritonPart [11] is a more recent open-source, constraints-driven partitioning tool for VLSI physical design. Another type of partitioner is based on spectral methods. HyperSF [26] and HyperEF [27] achieve more efficiency in the coarsening phase without losing the key structural properties of the original hypergraph. SpecPart [28] and K-SpecPart [29] can produce vertex embeddings and further reduce the cutsize of a given partitioning result, but at the cost of higher runtime and complexity.

2) *Parallel Hypergraph Partitioning*: As the scale of hypergraphs grows, the need for faster partitioning algorithms has led to the development of many parallel partitioners, like Zoltan [16], Parkway [17], Social Hash Partitioner [15] and Mt-KaHyPar series [19–21]. However, none of them can guarantee deterministic results, which is a critical requirement in some domains, like VLSI circuit design.

BiPart [18] is the first deterministic parallel hypergraph partitioner. It introduces parallelism in all three phases of the multilevel paradigm, and achieves good partitioning quality and runtime. However, BiPart fails to control the vertex

weights in the coarsening phase and ignores the up-to-date gains in the refinement phase, which leads to rather fluctuating results against its hyperparameters.

Mt-KaHyPar-SDet [22] is the deterministic version of Mt-KaHyPar, which outperforms BiPart regarding quality, runtime, scalability, and does not lose too much performance compared to its non-deterministic versions. It uses direct k -way partitioning and has better control over the vertex weights and partition balance throughout the whole partitioning process. The main drawback of Mt-KaHyPar-SDet is its parallel refinement method based on label propagation [30] disables its opportunity to consider the lower bound of the balance constraint in (2), which is not acceptable in some scenarios.

III. THE PROPOSED METHOD

In this section, we present the techniques in BlasPart. Firstly, a recursive multilevel bisection framework is proposed to perform k -way partitions with deterministic results. Within it, efficient techniques for bisection are adopted based on whether a task is executed serially or parallelly. Then, a level-dependent strategy is proposed to enforce the balance constraint to each bisection task. Finally, we analyze the scalability of BlasPart with respect to the number of partitions and threads.

A. Overall Framework

Although [31] states that recursive bisection often achieves substantially worse results than direct k -way, it becomes different under the parallel environment, especially when we are dealing with the problem of partitioning a large-scale hypergraph into a large number of balanced blocks. The reasons lie in both the initial and refinement phases.

In the initial phase, we need to find a balanced partition of the hypergraph, but underload is very likely to happen if we keep pursuing quality, and many algorithms actually still use recursive bisection here (or they do not focus on underload). Besides, during the k -way refinement, the definition of *cut-net* is unaware of the number of blocks a cut hyperedge intersects, and can not be eliminated unless all the vertices in the hyperedge are assigned to the same partition, which is usually very difficult when k is large. By contrast, recursive bisection does not have either of these problems.

Moreover, recursive bisection can be easily parallelized by first partitioning the hypergraph into two blocks, and then using different threads to recursively partition each block into two blocks until the desired number of blocks is reached. This is the main idea behind BlasPart, which is shown in Fig. 1.

To partition a hypergraph into k blocks using p threads (suppose $k > p$ and is the power of 2 for simplicity), BlasPart utilizes the divide and conquer strategy and produces a binary tree that can be separated into two main stages. The first stage includes the first $\log(p)$ layers of the tree, where each bisection task uses multiple threads. The second stage includes the rest $\log(k/p)$ layers of the tree, where each bisection task uses only one thread and is executed serially. The last layer consists of $k/2$ bisection tasks that eventually output the k partitions. This framework involves both serial and parallel bisection algorithms, which are detailed in the following part.

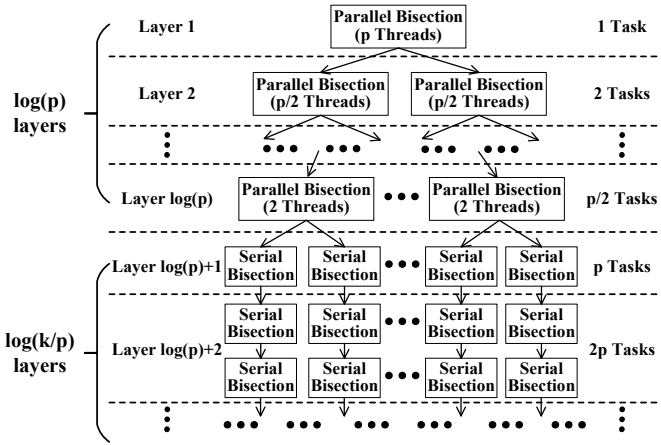


Fig. 1: The overall framework of BlasPart, for parallelly partitioning a hypergraph into k blocks using p threads.

B. Deterministic Bisection Algorithms

1) *Serial*: For the serial algorithm, we mostly follow the workflow of hMETIS [4]. In the coarsening phase, we use the first-choice (FC) scheme [32] by applying the heavy-edge score function to cluster highly connected vertices. In the initial partitioning phase, we use the greedy hypergraph growing (GHG) algorithm [33]. We combine the two gain functions (max-pin and max-net) with three vertex selecting strategies (sequential, global, round-robin) [8, 34] to generate multiple initial partitions, from which a few best will be eventually selected. In each level of the refinement phase, solutions with a cutsize 10% worse than the current smallest will be discarded. We use the greedy refinement [32] instead of the Fiduccia-Mattheyses (FM) algorithm [35], because during the whole recursive process, we will deal with hypergraphs of different sizes and structures, to which the speed of FM algorithm is quite sensitive. This is especially important because now we are concerned about the critical path of the whole partitioning process when a steady runtime is more important. The serial algorithm does not involve deterministic issues.

2) *Parallel*: For the parallel algorithm, because the serial coarsening phase is very time-consuming and lacks parallelism when the hypergraph is large, we replace it with the parallel one in [22]. This fully exploits the parallelism on the first $\log(p)$ layers of the binary tree in Fig. 1, while providing deterministic results at the same time. Because the hypergraphs in the initial partitioning phase are much smaller than the original one, we still use the serial GHG algorithm and utilize the parallelism in finding multiple diverse initial solutions, just like [22]. When collecting the results from the parallel bisection tasks, we sort them by their task ID before filtering to ensure determinism as well. Different from [22], we also apply this principle in the refinement phase. We use the same greedy refinement algorithm as the serial one, and refine different solutions in parallel. This enables the algorithm to keep more diversity to compensate for the quality loss in the coarsening phase. It also avoids the loss in the lack of up-to-date gains during the vertex moves. Determinism is also guaranteed because the order of the partitions is fixed.

C. The Balance Constraint for Bisection Tasks

When partitioning a large hypergraph, especially using the approach of recursive bisection, achieving good balance is a challenge. Some recursive bisection algorithms like hMETIS [4] and BiPart [18] only use a fixed value of imbalance factor, i.e. ϵ in (2), to enforce the balance constraint for every bisection task. In this case, in order to fulfill the overall balance constraint (2) for the k -way partitioning, the imbalance factor ϵ_i for solving the i -th layer bisection problem is set as:

$$\begin{aligned} \epsilon_i &= \min\{(1 + \epsilon)^{\frac{1}{\log(k)}} - 1, 1 - (1 - \epsilon)^{\frac{1}{\log(k)}}\} \\ &= (1 + \epsilon)^{\frac{1}{\log(k)}} - 1, \quad \forall 1 \leq i \leq k - 1. \end{aligned} \quad (3)$$

However, this is often too strict for large k , leading to the actual imbalance ratio δ of the result being much smaller than the imbalance factor ϵ . As a result, this approach unavoidably causes a decrease in the quality of partitioning.

In this work, we propose a different strategy for setting the imbalance factor for the recursive bisection approach, which produces the partitioning result with better quality and good balance at the same time. Firstly, we see that (3) infers that at the i -th layer of bisection, the enforced constraint includes:

$$|V_j^{(i)}| \leq (1 + \epsilon)^{\frac{1}{\log(k)}} \cdot \frac{|V^{(i)}|}{2}, \quad j = 1, 2, \quad (4)$$

where $|V^{(i)}|$ denotes the number of nodes of the hypergraph for performing bisection. At the very bottom layer, i.e. layer $\log(k)$ in Fig. 1, the upper bound in (4) for a block's number of nodes would be about $(1 + \epsilon)^{\frac{1}{\log(k)}} \frac{|V|}{2^{\log(k)}} = (1 + \epsilon)^{\frac{1}{\log(k)}} \frac{|V|}{k}$, if the partitions are all well balanced and $|V^{(i)}| \approx \frac{|V|}{2^{(i-1)}}$. This upper bound is too small, compared to that we want to enforce, i.e. $(1 + \epsilon) \frac{|V|}{k}$ in (2). Therefore, our idea is to relieve this upper bound, and make it depend on the layer of bisection. We change the upper bound of constraint to:

$$|V_j^{(i)}| \leq (1 + \epsilon)^{\frac{1}{\log(k) - i + 1}} \cdot \frac{|V|}{2^i}, \quad j = 1, 2, \quad (5)$$

for bisection tasks of the i -th layer. In this way, at the bottom layer (i.e. $i = \log(k)$) the upper bound is the same as that in (2). It guarantees that the overall balance constraint on the upper bound is satisfied. At the upper layers of bisection, the constraint becomes gradually tight; at the first layer, it is the same as (4), i.e. that used in the conventional approach.

Similar analysis and manipulation are made on the lower bound of the constraint. As a result, we set the lower bound of the balance constraint for the i -th layer of bisection to:

$$|V_j^{(i)}| \geq (1 - \epsilon)^{\frac{1}{\log(k) - i + 1}} \cdot \frac{|V|}{2^i}, \quad j = 1, 2. \quad (6)$$

With the constraints (5) and (6) set for the bisection tasks, the final partitioning result will satisfy the balance constraint (2) if all the bisection problems can be solved. The exception occurs when $|V^{(i)}|$ in (4) is far from $\frac{|V|}{2^{(i-1)}}$, which may cause the absolute imbalance margins in (5) and (6) making the optimization problem infeasible. We have made extensive experiments and found that this situation happens in very low probability. If it does happen, which only includes two

types of situations: the average number of vertices $\frac{|V_1^{(i)}|+|V_2^{(i)}|}{2}$ is smaller than the lower bound, or bigger than the upper bound, we will force an even balance from this task to all its descendants to reduce the violation as much as possible. The results show that we can control the imbalance in an acceptable range. If we really need a strict balance solution, we can just set all upper and lower bounds according to (3).

We also utilize some other strategies to further optimize the balance and reduce the risk of unsolvability. One hidden strength of the recursive bisection is that after each partitioning task, the number of hyperedges decreases because the cut hyperedges in the last layer are discarded since they do not contribute to the cutsize of its descendants, according to the definition of `cut-net`. As a result, as the partitioning goes deeper, many isolated vertices emerge. They do not belong to any hyperedges and can be easily assigned to any partition without affecting the cutsize. Therefore, we ignore these vertices during the whole bisection process, and after the rest vertices are assigned, we assign these vertices to the partition with the smaller size. This is a very simple yet effective way to balance the partition size. Another strategy is to consider balance as a criterion in filtering the solutions in both the initial and refinement phases. When two solutions have the same cutsize, we break the tie by choosing the one with better balance. This criterion can also be considered during the greedy refinement phase, where we can choose the move that leads to better balance when the gain is the same, and accept moves with zero gain but can improve the balance.

D. Scalability Analysis

BlasPart aims at partitioning large hypergraphs into a large number of blocks in parallel. So, both the scalabilities in terms of the number of blocks and the number of threads are of concern. Separating hypergraphs according to the bisection results occupies very little time in the whole partitioning process, so we do not consider it in the following analysis.

When the number of blocks (k) increases, we analyze the runtime (t) breakdown by separating t according to the correlation with k :

$$t = f(k) + C, \quad (7)$$

where $f(k)$ varies along with k , and C remains constant when k changes. The reason for the poor scalability of BiPart is that its C equals 0 and the high scaling rate of $f(k)$ because it uses all threads in each bisection task.

Mt-KaHyPar-SDet improves this by using the direct k -way partitioning, which avoids the repetitive calling of the partitioning algorithm. Comparing the three phases of the multilevel paradigm, it is worth noting that when doing the k -way partitioning, the coarsening phase is obviously different from the other two, because its runtime does not depend on k . k will only influence the maximum allowed cluster size, but this will only happen at the last few levels when the coarsened hypergraph is small enough and does not consume much time. Based on the analysis above, C of Mt-KaHyPar-SDet is actually the coarsening runtime, and $f(k)$ consists of

the initial partitioning and refinement runtime. The invariant coarsening runtime C and lower scaling rate of $f(k)$ are the main reasons for its better scalability than BiPart.

Compared to Mt-KaHyPar-SDet, the special point of BlasPart is that its C is actually the whole first stage (the first $\log(p)$ layers in Fig. 1), because as k increases, the only differences in the first stage are the upper and lower bounds, which will hardly affect its runtime. Considering the second stage, BlasPart has an even lower scaling rate of $f(k)$ than Mt-KaHyPar-SDet, because although the number of tasks doubles in each layer, the sizes of the hypergraphs also decrease by half respectively. Considering the hypergraphs involved in this stage are more than p times smaller than the original one, the total runtime of each layer increases very slowly, and may even result in a decrease of the total runtime on some hypergraphs when k doubles. Another incidental advantage is that other optimizations like V-cycle [4] and COCP [11] can be added in each bisection task in the last few layers to further improve the quality with little time cost, but we do not further discuss it in this paper due to space limitation.

According to Fig. 1, each bisection task in the second stage uses one thread, which involves no communication cost. Its runtime also differs little from each other, because the hypergraphs in here are small. Therefore, the scalability of BlasPart in terms of the number of threads (p) in this stage is good. However, the first stage is usually the bottleneck, especially when the number of blocks is relatively small. Because the runtime difference between tasks in the first stage is quite sensitive to the input hypergraph, which further affects the runtime critical path of the binary tree, the overall scalability of BlasPart (in terms of p) may vary on different benchmarks. Nevertheless, efficient techniques that are insensitive to the hypergraph structure have been selected (as stated in Section III-B), to avoid this problem as much as possible.

IV. EXPERIMENTAL RESULTS

In this section, we evaluate the proposed BlasPart by comparing it with state-of-the-art partitioners on two sets of benchmarks. BlasPart is implemented in C++ and utilizes Intel's tbb library for parallelization. We first compare BlasPart with the newest hMETIS (v2.0 pre1¹) which supports direct k -way partition and the complete balance constraint (2). hMETIS is a serial-computing partitioner, the SOTA of partitioning quality. Then, we compare BlasPart with the parallel partitioners Mt-KaHyPar-SDet² [22] and BiPart³ [18] on larger benchmarks. All experiments are conducted on a Ubuntu 22.04 Linux server with AMD EPYC 7V12 64-Core CPU and 512GB RAM.

A. Comparison with hMETIS

The comparison with hMETIS is made on the Titan23 benchmark suite [36], which contains hypergraphs from 22 circuits covering a wide range of applications. For hMETIS, all

¹<http://glaros.dtc.umn.edu/gkhome/views/metis>

²<https://github.com/kahypar/mt-kahypar>

³<https://github.com/IntelligentSoftwareSystems/Galois/tree/master/lonestar/analytics/cpu/bipart>

parameters are set to their default values. Because $nruns = 10$ in hMETIS, we also run BlasPart 10 times with different seeds as well, and calculate the best cutsize and average runtime of both partitioners. Fig. 2 shows the results for $\epsilon = 0.1$ and $k = 16$. BlasPart uses 16 threads and its cutsize and runtime are normalized by those obtained by hMETIS, respectively.

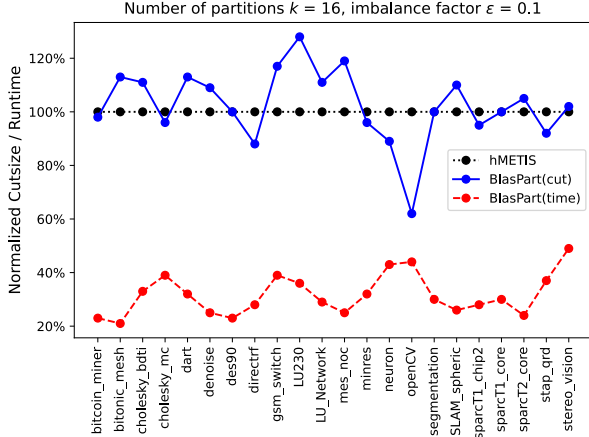


Fig. 2: Normalized cutsize and runtime of BlasPart on the Titan23 benchmark suite, with respect to those of hMETIS.

From Fig. 2, we can see that BlasPart outperforms hMETIS on cutsize on 9 of the 22 benchmarks. After statistics, we find out that the cutsize of BlasPart is just 2% larger than that of hMETIS on average, which shows that our parallel BlasPart maintains competitive partitioning quality compared to the k -way serial partitioner hMETIS. As for the runtime, although all the benchmarks are relatively small (with 90K-1M vertices), which indicates the optimization space is quite limited, BlasPart still exhibits a $3.33\times$ speedup on average, and a maximum $4.70\times$ speedup (on the *bitconic_mesh*). Besides, both the results of hMETIS and BlasPart fulfill the balance constraint with the measured imbalance ratio δ all within 0.1. This validates the balance strategy proposed in Section III-C.

B. Comprasion with Parallel Partitioners on Large Cases

TABLE I: Characteristics of six large benchmarks.

Benchmark	#Vertices	#Hyperedges	#Pins
WB	9,845,725	6,920,306	57,156,537
NLPK	3,542,400	3,542,400	96,845,792
SAT	13,378,010	521,147	39,203,144
TITAN	1,089,284	1,448,151	4,557,197
DAC	1,360,217	1,340,418	4,931,418
IND	10,904,226	1,327,578	21,535,851

To comprehensively evaluate the performance of BlasPart on large-scale hypergraphs, we conduct the following experiments on six large benchmarks listed in Table I. The number of pins is the sum of the cardinalities of all hyperedges. WB and NLPK are from the SuiteSparse Matrix Collection [37]. SAT is from the 2014 SAT Competition [14]. TITAN and DAC are two VLSI instances from the Titan23 benchmark suite [36] and DAC 2012 Routability-Driven Placement Contest [38], respectively. They all are the largest hypergraphs in the corresponding benchmark suite. IND is a large hypergraph we obtained from an industrial circuit design. The cases WB,

NLPK, and SAT were also used in the experiments of BiPart [18] and Mt-KaHyPar-SDet [22]. In our experiment, we set $\epsilon = 0.1$ and vary k from 16 to 4096, and all the partitioners run with 16 threads and their default parameter settings.

We first compare BlasPart with Mt-KaHyPar-SDet. The imbalance ratios (δ calculated with (1)) of their results are listed in Table II, while the results of cutsize and runtime are shown in Fig. 3. From Table II we can see that Mt-KaHyPar-SDet largely violates the balance constraint in most cases, because it only considers the upper bound of the partition size. In contrast, BlasPart fulfills the balance constraint in most cases. The only exception is SAT, where the imbalance ratio of BlasPart's result is 0.179 when $k = 1024$. It still corresponds to a partitioning with a relatively good balance. The results in Table II validate the proposed balance strategy, which relaxes the balance constraint but still achieves results with good balance on large-scale hypergraph partitioning.

In Fig. 3, the normalized cutsizes obtained with BlasPart are displayed with those of Mt-KaHyPar-SDet set as the baseline. We can see that BlasPart achieves similar cutsizes to Mt-KaHyPar-SDet and overall outperforms it when k is large enough. In terms of runtime, although BlasPart generally consumes more time than Mt-KaHyPar-SDet when k takes small values like 16, 32, and 64, it gradually becomes more efficient than the latter when k increases. This is consistent with our analysis in Section III-D. The runtime of Mt-KaHyPar-SDet grows approximately exponentially with $\log(k)$ (linearly with k), while the runtime of BlasPart grows nearly linearly with $\log(k)$ (logarithmically with k). The steady runtime C of the first stage and fast speed in the second stage in BlasPart are the main reasons. It eventually reaches a maximum of $4.69\times$ and an average of $3.33\times$ speedup over Mt-KaHyPar-SDet on the six benchmarks when $k = 4096$. In general, for the balanced large-scale hypergraph partitioning, BlasPart achieves better balance and scaling of runtime while maintaining competitive partitioning quality compared to Mt-KaHyPar-SDet.

BlasPart is also compared with another deterministic parallel partitioner BiPart, which can support the complete balanced constraint (2). The experiments in [22] demonstrate that Mt-KaHyPar-SDet has better performance than BiPart in terms of runtime and cutsize. Our experiments confirm this. The detailed comparison results on case DAC are listed in Table III as an example. From it, we can see that BlasPart achieves significant improvements on the cutsize and runtime, compared to BiPart. As for the imbalance ratio, both partitioners satisfy the constraint with $\epsilon = 0.1$. Because BiPart employs (3) for setting the constraint for each bisection problem, it obviously degrades its partition quality due to this too-strict setting.

Lastly, an experiment is done to show the parallel scalability of BlasPart. We set $\epsilon = 0.1$ and $k = 1024$ on all benchmarks, and Fig. 4 shows the speedup of BlasPart along with the number of threads selected from $\{1, 2, 4, 8, 16, 32\}$. As we have mentioned in Section III-D, the scaling performance of BlasPart may vary on different benchmarks. BlasPart achieves the best scaling on NLPK, with the parallel speedup of $6.25\times$ using 16 threads, but performs relatively poorly on SAT

TABLE II: Imbalance ratios of k partitions produced by Mt-KaHyPar-SDet and BlasPart on large cases (setting $\epsilon = 0.1$).

k	WB		NLPK		SAT		TITAN		DAC		IND	
	Mt-KaHyPar-SDet	BlasPart	Mt-KaHyPar-SDet	BlasPart	Mt-KaHyPar-SDet	BlasPart	Mt-KaHyPar-SDet	BlasPart	Mt-KaHyPar-SDet	BlasPart	Mt-KaHyPar-SDet	BlasPart
16	0.100	0.092	0.277	0.074	0.209	0.091	0.152	0.100	0.244	0.099	0.226	0.081
32	0.100	0.100	0.498	0.080	0.231	0.093	0.277	0.092	0.285	0.097	0.196	0.077
64	0.100	0.099	0.552	0.094	0.231	0.098	0.261	0.095	0.390	0.098	0.381	0.087
128	0.100	0.098	0.378	0.089	0.449	0.100	0.515	0.098	0.311	0.100	0.294	0.086
256	0.100	0.099	0.566	0.097	0.669	0.131	0.644	0.100	0.370	0.099	0.505	0.098
512	0.100	0.099	0.803	0.095	0.545	0.100	0.556	0.100	0.394	0.099	0.379	0.091
1024	0.100	0.100	0.998	0.100	0.449	0.179	0.634	0.099	0.365	0.100	0.445	0.095
2048	0.100	0.100	0.947	0.100	0.473	0.173	0.551	0.098	0.418	0.100	0.424	0.098
4096	0.100	0.100	0.763	0.100	0.480	0.139	0.568	0.098	0.345	0.100	0.421	0.099

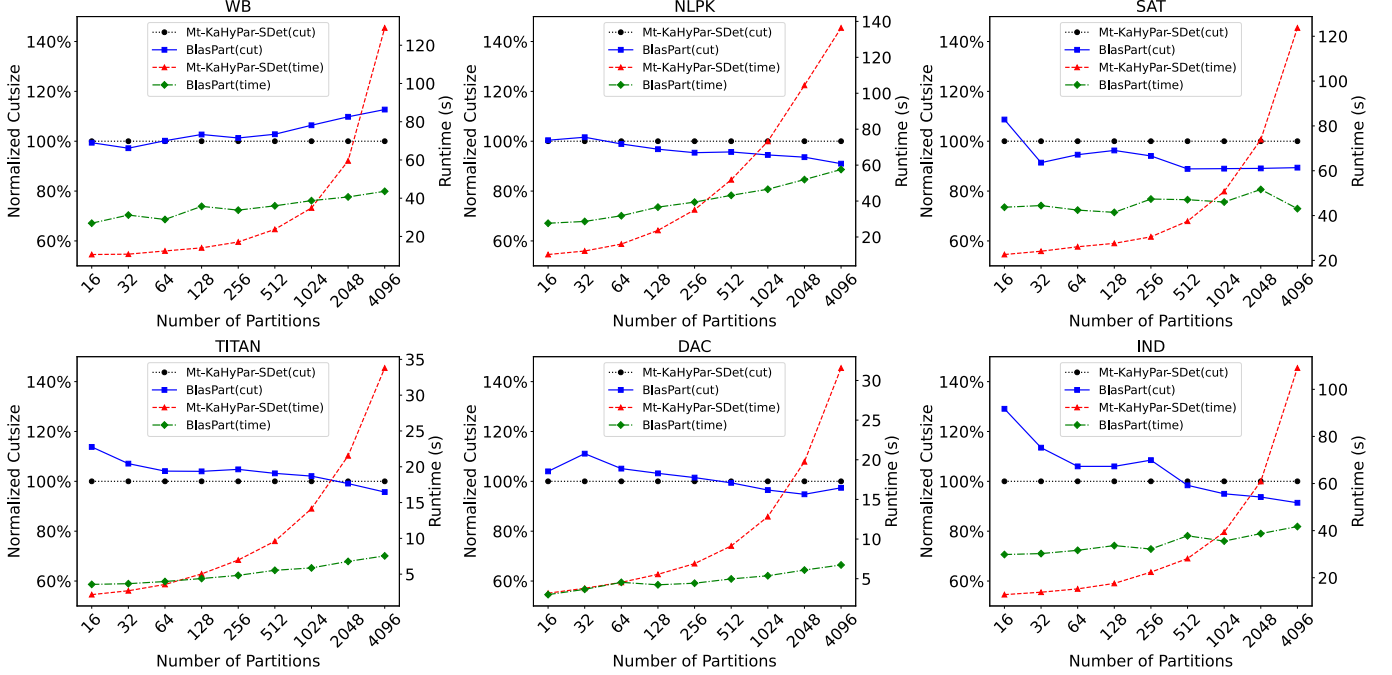


Fig. 3: Normalized cutsize and actual runtime for Mt-KaHyPar-SDet and BlasPart on the large benchmarks (16 threads).

TABLE III: Performance of BiPart and BlasPart on the DAC benchmark, setting $\epsilon = 0.1$ (16 threads).

k		16	32	64	128	256	512	1024	2048	4096
BiPart	cutsize	55049	83633	119424	142369	168388	196320	239393	298877	343686
	runtime(s)	10.17	14.29	22.22	29.82	42.68	54.72	66.42	75.74	97.70
	imbalance	0.087	0.061	0.092	0.080	0.085	0.067	0.086	0.081	0.078
BlasPart	cutsize	30219	47714	65265	83856	98104	115327	134225	154135	174936
	runtime(s)	2.99	3.66	4.54	4.23	4.43	4.97	5.37	6.09	6.73
	imbalance	0.099	0.100	0.099	0.098	0.099	0.099	0.100	0.100	0.100

because of its great disparity in the number of vertices and hyperedges. The parallel speedup of BlasPart is also limited on TITAN and DAC, because of their relatively small sizes.

V. CONCLUSION

In this paper, we propose a parallel hypergraph partitioner BlasPart, which is designed for partitioning a large-scale hypergraph into a large number of balanced blocks. BlasPart employs a two-stage recursive bisection approach, which can give partitioning results with good balance and high quality. We introduce a novel strategy for setting the balance constraint in the recursive bisection, which is more relaxed than the conventional approach and can adapt to different hypergraphs. We also utilize some other strategies to further optimize the

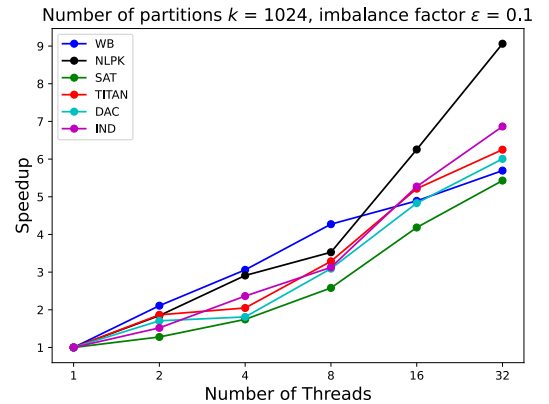


Fig. 4: Parallel speedup of BlasPart on different benchmarks.

balance and reduce the risk of unsolvability. Experimental results show that BlasPart achieves competitive partitioning quality compared to the state-of-the-art partitioners, and it is more efficient and scalable on large-scale hypergraph partitioning. In the future, we will extend BlasPart to support any number of partitions and introduce more techniques in the whole framework to further improve the partitioning quality.

REFERENCES

- [1] D. S. Johnson and M. R. Garey, *Computers and intractability: A guide to the theory of NP-completeness*. WH Freeman, 1979.
- [2] K. Andreev and H. Räcke, “Balanced graph partitioning,” in *Proceedings of the sixteenth annual ACM symposium on Parallelism in algorithms and architectures*, 2004, pp. 120–124.
- [3] T. Lengauer, *Combinatorial algorithms for integrated circuit layout*. Springer Science & Business Media, 2012.
- [4] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, “Multilevel hypergraph partitioning: Applications in VLSI domain,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 7, no. 1, pp. 69–79, 1999.
- [5] Ü. V. Çatalyürek and C. Aykanat, “PaToH (partitioning tool for hypergraphs).” 2011.
- [6] U. V. Catalyurek, M. Deveci, K. Kaya, and B. Uçar, “UMPa: A multi-objective multi-level partitioner for communication minimization,” *Graph partitioning and graph clustering*, vol. 588, p. 53, 2013.
- [7] H. Leitgeb, “HYPE: A system of hyperintensional logic (with an application to semantic paradoxes),” *Journal of Philosophical Logic*, vol. 48, no. 2, pp. 305–405, 2019.
- [8] S. Schlag, T. Heuer, L. Gottesbüren, Y. Akhremtsev, C. Schulz, and P. Sanders, “High-quality hypergraph partitioning,” *ACM Journal of Experimental Algorithmics*, vol. 27, pp. 1–39, 2023.
- [9] C. J. Alpert, “The ISPD98 circuit benchmark suite,” in *Proceedings of the 1998 international symposium on Physical design*, 1998, pp. 80–85.
- [10] A. E. Caldwell, A. B. Kahng, A. A. Kennings, and I. L. Markov, “Hypergraph partitioning for VLSI CAD: methodology for heuristic development, experimentation and reporting,” in *Proceedings of the 36th annual ACM/IEEE Design Automation Conference*, 1999, pp. 349–354.
- [11] I. Bustany, G. Gasparyan, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, “An open-source constraints-driven general partitioning multi-tool for VLSI physical design,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [12] B. Li, S. Bi, H. You, Z. Qi, G. Guo, R. Sun, and Y. Zhang, “MaPart: An efficient multi-FPGA system-aware hypergraph partitioning framework,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [13] Ü. V. Catalyurek and C. Aykanat, “Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication,” *IEEE Transactions on parallel and distributed systems*, vol. 10, no. 7, pp. 673–693, 1999.
- [14] A. Belov, D. Diepold, M. Heule, and M. Järvisalo, “The SAT competition 2014,” in *Proc. SAT COMPETITION*, 2014, p. 45.
- [15] I. Kabiljo, B. Karrer, M. Pundir, S. Pupyrev, A. Shalita, A. Presta, and Y. Akhremtsev, “Social hash partitioner: a scalable distributed hypergraph partitioner,” *arXiv preprint arXiv:1707.06665*, 2017.
- [16] K. D. Devine, E. G. Boman, R. T. Heaphy, R. H. Bisseling, and U. V. Catalyurek, “Parallel hypergraph partitioning for scientific computing,” in *Proceedings 20th IEEE International Parallel & Distributed Processing Symposium*. IEEE, 2006, pp. 10–pp.
- [17] A. Trifunović and W. J. Knottenbelt, “Parallel multilevel algorithms for hypergraph partitioning,” *Journal of Parallel and Distributed Computing*, vol. 68, no. 5, pp. 563–581, 2008.
- [18] S. Maleki, U. Agarwal, M. Burtscher, and K. Pingali, “Bipart: a parallel and deterministic hypergraph partitioner,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 161–174.
- [19] L. Gottesbüren, T. Heuer, P. Sanders, and S. Schlag, “Scalable shared-memory hypergraph partitioning,” in *23rd Workshop on Algorithm Engineering and Experiments (ALENEX 2021)*. SIAM, 2021, pp. 16–30.
- [20] L. Gottesbüren, T. Heuer, P. Sanders, and S. Schlag, “Shared-memory n -level hypergraph partitioning,” in *24th Workshop on Algorithm Engineering and Experiments (ALENEX 2022)*. SIAM, 01 2022.
- [21] L. Gottesbüren, T. Heuer, and P. Sanders, “Parallel flow-based hypergraph partitioning,” in *20th International Symposium on Experimental Algorithms (SEA 2022)*, vol. 233. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, pp. 5:1–5:21.
- [22] L. Gottesbüren and M. Hamann, “Deterministic parallel hypergraph partitioning,” in *European Conference on Parallel Processing*. Springer, 2022, pp. 301–316.
- [23] M. Deveci, K. Kaya, B. Uçar, and Ü. V. Çatalyürek, “Hypergraph partitioning for multiple communication cost metrics: Model and methods,” *Journal of Parallel and Distributed Computing*, vol. 77, pp. 69–83, 2015.
- [24] A. Trifunovic and W. J. Knottenbelt, “Parkway 2.0: A parallel multilevel hypergraph partitioning tool,” in *International Symposium on Computer and Information Sciences*. Springer, 2004, pp. 789–800.
- [25] C. J. Alpert and A. B. Kahng, “Recent directions in netlist partitioning: A survey,” *Integration*, vol. 19, no. 1-2, pp. 1–81, 1995.
- [26] A. Aghdaei, Z. Zhao, and Z. Feng, “HyperSF: Spectral hypergraph coarsening via flow-based local clustering,” in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [27] A. Aghdaei and Z. Feng, “HyperEF: Spectral hypergraph coarsening by effective-resistance clustering,” in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.
- [28] I. Bustany, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, “SpecPart: A supervised spectral framework for hypergraph partitioning solution improvement,” in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.
- [29] I. Bustany, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, “K-SpecPart: Supervised embedding algorithms and cut overlay for improved hypergraph partitioning,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [30] H. Meyerhenke, P. Sanders, and C. Schulz, “Parallel graph partitioning for complex networks,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 9, pp. 2625–2638, 2017.
- [31] H. D. Simon and S.-H. Teng, “How good is recursive bisection?” *SIAM Journal on Scientific Computing*, vol. 18, no. 5, pp. 1436–1445, 1997.
- [32] G. Karypis and V. Kumar, “Multilevel k -way hypergraph partitioning,” in *Proceedings of the 36th annual ACM/IEEE design automation conference*, 1999, pp. 343–348.
- [33] G. Karypis and V. Kumar, “A fast and high quality multilevel scheme for partitioning irregular graphs,” *SIAM Journal on scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.
- [34] T. Heuer, “Engineering initial partitioning algorithms for direct k -way hypergraph partitioning,” Ph.D. dissertation, Karlsruher Institut für Technologie (KIT), 2015.
- [35] C. M. Fiduccia and R. M. Mattheyses, “A linear-time heuristic for improving network partitions,” in *Papers on Twenty-five years of electronic design automation*, 1988, pp. 241–247.
- [36] K. E. Murray, S. Whitty, S. Liu, J. Luu, and V. Betz, “Titan: Enabling large and complex benchmarks in academic CAD,” in *2013 23rd International Conference on Field programmable Logic and Applications*. IEEE, 2013, pp. 1–8.
- [37] T. A. Davis and Y. Hu, “The University of Florida sparse matrix collection,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 38, no. 1, pp. 1–25, 2011.
- [38] N. Viswanathan, C. Alpert, C. Sze, Z. Li, and Y. Wei, “The DAC 2012 routability-driven placement contest and benchmark suite,” in *Proceedings of the 49th Annual Design Automation Conference*, 2012, pp. 774–782.